Codex 核心工作流与 MiniCodex

前言

大约自 2025 年以后随着各种大语言模型智能体代码工具的蓬勃发展，Vibe Coding 这一概念也随之广泛流行起来。在实践上它已经能够有效加速很多代码工作，我本身也是其重度用户，并且经常使用 OpenAI Codex 来辅助日常工作。

Codex 就是几个最流行的 coding agent 之一，同时更棒的是它本身的代码完全开源 (准确来说是只有 Codex cli 版本开源)。当前我也正在学习 Codex 的源码。但原 Codex 代码已经高度封装化，和工程化，这对于实际使用来说自然相当重要，但同时却也会使得直接学习其工程内容变得困难。其核心实现原理的简洁与美妙也往往被这些繁杂的工程实现所掩盖。我因此想到或许可以尝试从零开始编写一个最简化的 mini-codex，尝试用最少的代码复刻出 codex 的主要功能。我于是做了这个小玩具，并围绕它做了一些简单的研究工作。

本文当中我们就会介绍 mini-codex 的各种实现细节以及针对它的部分实验，相信对于熟悉智能体工作的朋友而言都完全不会感到陌生。mini-codex 本身在技术实现上其实没有什么难度，不过却能够看到很多有趣的实验现象。后续我也可能会继续推出 OpenAI Codex 源码解析系列的博客，并对 mini-codex 做更多后续的更新与实验研究。

智能体循环

首先我们直接来看 mini-codex 的核心工作逻辑，它在源代码当中被命名为 `run_turn` (`run_agent_loop`) (在 OpenAI Codex 源码当中核心工作循环的名称同样也是 `run_turn`)。当前在 mini-codex 当中这个函数的实现方法就是 (经过删减仅保留主要逻辑)

```
fn run_agent_loop(&mut self) -> Result<()> {
    for _ in 0..LOOP_LIMIT {
        self.compact_history_if_needed(CompactionMode::MidTurn)?;
        let messages = build_messages(&self.config.workspace_root,
&self.history.entries);
        let reply = call_model(&self.client, &self.config.llm, messages, true)?;
        let assistant_tool_calls = reply
            .tool_calls
            .iter()
            .map(|call| AssistantToolCall {
                id: call.id.clone(),
                name: call.name.clone(),
                arguments: call.arguments.clone(),
            })
            .collect();
```

```
        self.history
            .push_assistant(reply.content.clone(), assistant_tool_calls);
        if !reply.tool_calls.is_empty() {
            self.handle_tool_calls(reply.tool_calls)?;
            self.save_history()?;
            continue;
        }

        return Ok(());
    }

    Err(anyhow!(
        "agent loop exceeded {LOOP_LIMIT} steps without producing a final
response"
    ))
}
```

在这个 coding agent 的工作流当中，一次用户交互就被称作一个 turn。其中每一个 turn 当中 mini-codex 可做的事情都仅有两类，执行 shell 指令，或者直接回复。对于用户的每一个指令我们都会循环迭代这一过程直到 mini-codex agent 不再输出 `tool_calls`，而这通常就意味着工作结束。

虽然当前的 mini-codex 当中也有千余行代码，但其中绝大多数都是用于实现 TUI 和 LLM Client 的辅助性代码，实际核心部分可能仅有三四百行。这里面就包括 shell_tool 的实现，以及简易的记忆机制等等。

主要技术

mini-codex 的主要技术简而言之就是没什么技术。。。当然参考原 OpenAI Codex 的实现看来，当前智能体应用的构建上确实也就是这样简单的做法才最有效。

工具调用

唯一的工具：shell_tool

当前版本的 mini-codex (v0.1.0) 有且仅有唯一的一个必选工具，`shell_tool`。顾名思义它赋予了 mini-codex 执行各类 shell 代码的能力。其实现也非常简洁，我们会让后端大模型在调用这一工具时直接提供原始的 shell 代码，在获取用户许可后直接解释执行并获取输出。可以说这就是 mini-codex 全部能力的核心所在。

此外必须指出的是这里的 `shell_tool` 实现存在明显的安全性问题，由于我们对于 mini-codex 所能够执行的代码几乎不做限制 (只在 prompt 当中给了一定的要求)，它就完全有可能做出危害性的举动。想要解决这一问题可能需要大量的工程方案，但它与 mini-codex 的核心机制无关，为代码简洁性考虑我就没有做这方面的工作了。也因此实际上并不推荐在工作中实际使用 mini-codex (当然实践当中它做出危险举动的概率确实并不高)。

在原 OpenAI 的 Codex 当中有一个非常常用的工具，`apply_patch`，可以方便地对代码进行编辑修改。它的实现也并不复杂，但是我们的 mini-codex 并没有实装这一工具。实际上即使没有 `apply_patch`，实践当中 mini-codex 仍可以通过 `shell_tool` 使用 `cat`，`sed`，或者执行 python 脚本等去实现源代码编辑的功能。这在速度和 UI 展示上或许会略差于原 Codex 的方案，但在编辑操作上却具有明显更高的自由度。实际上 shell 指令本身就已经是极大量工具的集合，是否增加 `apply_patch` 这一工具并不会产生什么本质的改变，因此同样出于简洁性的考虑我就略去了相关的实现。

工具调用的具体实现

我们在 mini-codex 的工具和记忆实现上都遵循了智能体工具调用的一般方法，也就是通过 API 调用当中的 `tools` 参数传入 `shell_tool` 的描述和相关定义，然后通过解析 API 返回的结构化工具调用结果来在本地执行。具体实现可以参见 `llm.rs` 当中的代码。这其实更多是一个规范化上的事情，并不算本质。

当我们通过 API 调用将工具相关的定义传输给 vllm, sglang 或者各家的私有语言模型推理引擎后，它们所做的事情也是将 `tools` 重新解析为文本并以特定格式插入到 prompt 当中 (其中可能会附带一些特殊 token)。很多时候它其实也就是在 system prompt 下自动插入了如下这段文本 (参见 sglang 当中 DeepSeek-V3.2 的文本格式模板 tool_chat_template_deepseekv32.jinja)

```
## Tools
You have access to the following tools:

### Tool 1
Description: ...
Parameters: ...
```

此外值得指出的是，当前推理模型是否启用深度思考，以及思考强度参数也都是通过 prompt 方法去实现的。在完成文本格式化并从语言模型中采样生成后，推理引擎会另外从语言模型的回复当中提取出结构化的工具调用信息。这一过程对于很多模型来说也都有所不同，其中同样以 DeepSeek-V3.2 为例，其在 sglang 当中的工具调用的提取脚本就在 deepseekv32_detector.py。

因此走 `tools` 接口的主要意义就在于让推理引擎后端帮我们处理了一大堆麻烦的 prompt 格式化以及解析问题，但这在工程实践当中其实也是很必要的。由于各家大模型使用的 prompt 格式化方案都不统一，如果要手动编写代码处理的话就会非常麻烦。与此同时由于各家大模型都针对着特定的输入格式进行训练，如果不保持 prompt 格式正确的话就很可能造成解析错误或者性能下降的情况。因此在构建智能体的过程中还是使用 `tools` 这类标准接口最好，就像使用 `chat/completions` 以及标准的 messages 格式一样。

记忆机制

我们在 mini-codex 当中同样使用的是最简单的记忆存储与压缩机制，它实际上就与 OpenAI Codex 当中的实现类似。在记忆存储上我们基本上就是直接存储了每一次用户和 agent 交互的 messages 信息，包括 `system`，`user`，`assistant`，`tool` 几类。同时稍微有所不同的是我们还额外对每个条目加入了 token 数量的统计，它通过对每次 API 调用时的结果统计得到，并且主要服务于长程任务下的记忆压缩。Codex 当中主要的短期记忆 (单 session 内部记忆) 机制基本上也是如此实现的。其长期记忆 (跨 session 记忆) 机制暂时没有做进 mini-codex 当中，暂时就先不做讨论。尽管它的实现实际上也并不算复杂。

在记忆存储这方面其实值得说的东西不多，而这里面的很多设计实际上都是为了记忆压缩服务的。记忆压缩的意义就是当语言模型处理的任务长度超出其上下文长度限制时，语言模型可以总结压缩此前的对话内容，以此取得更长的有效运行时间。它同时也有节约成本的作用 (上下文长度增加就会导致模型推理成本的增长)。

在 Codex 当中短期记忆相关的实现都放在 `codex/codex-rs/core/src/` 目录下的 `codex.rs` 和 `compact.rs` 两个源文件下，我们可以先来看一下它的实现方式。Codex 当中的上下文记忆压缩显然存在两种情况，pre-turn 和 mid-turn。其中 pre-turn 压缩会在 Codex 智能体行动之前上下文已超限，或者用户主动要求压缩时触发，而 mid-turn 则是在智能体行动过程当中超限时触发。它们两者在处理上会有一定的差别，主要就体现在 mid-turn 压缩时需要保持最后一条用户指令完整，并且提供完整的指示让智能体拿到压缩的记忆之后能够继续完成工作。

在触发压缩后 Codex 基本的工作流程就是：

1. 在当前的对话记录后插入一条临时的压缩 prompt 作为用户指令 (具体 prompt 参见 `codex/codex-rs/core/templates/compact/prompt.md`)。
2. 直接继续调用 API 进行补全，此时大模型会遵照这条临时用户指令将此前的对话记录进行压缩。
3. 提取这一条压缩的结果并替换掉此前的历史记录，并重新计算 token 使用量。
4. (如有需要) 再次重新插入一些关键的 context 信息，总结结果的 prefix，以及用户未完成的最后一条指令信息。

可以看到整个过程还是非常朴素且符合直觉的。在 mini-codex 的实现当中我们也基本上遵从这套方案，同时在实现上做了一些小巧思。我们在插入总结 prompt 并得到结果之后并不会替换掉历史记录，而是在历史记录的格式化过程当中加了一些额外的处理，使得我们的大模型始终只能看到第一条 system prompt (我们的初始 prompt) 和最后一条 system prompt (压缩后的历史信息) 及之后的信息，而两条 system prompt 中间的结果则始终对于 agent 不可见。如此我们实际上按照运行顺序完整存储了所有的历史信息，并且也很好地实现了压缩的功能。

mini-codex 的相关实验

在实现 mini-codex 后我也对它进行了一些实验，尤其关注了其与原版 OpenAI Codex 的能力对比。我在 mini-codex-examples 这个 GitHub 仓库当中放了原版 codex 和 mini-codex 制作的小网页游戏项目各三个。可以看到在这种不太长的常规测试项目当中，所实现的工程项目结果更多依赖于基础模型的能力，而这两个编码智能体框架所能带来的差异并不大。尽管如此，这一简单的 agent 框架仍是必要的，否则如果大模型并不能自主读取以及编辑项目文件，它自然也就无从自动完成这些开发工作了。

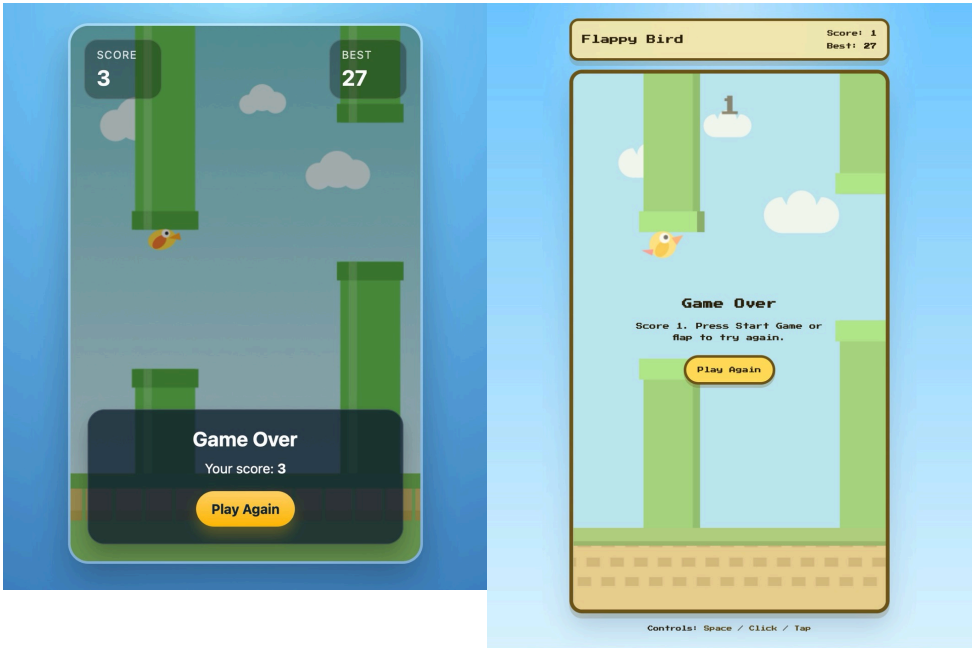

Figure 1: mini-codex (左) 和 OpenAI Codex (右) 在相同 prompt 下制作的 flappybird 游戏对比。更多示例和可运行版本的代码请参见 mini-codex-examples。

除此之外，实际上最初版的 mini-codex 也是通过 Codex 自动开发完成的小项目，但目前版本的 mini-codex 也已经可以一定程度上实现自举。我有尝试过几次使用 mini-codex 去修改自己的代码，修复 BUG 或者实现新功能，目前看来它都可以完成得很好。但是显然，这些都只是非常局限的孤立实验，很难以全面地评判各类 coding agent 的能力。

mini-codex 当前并没有实现类似 Codex 当中的 long-term memory 机制，也即跨 session 继承的记忆。它对于处理一些巨大的代码仓库而言可能会是非常有意义的。在后续的更新当中我们可能也会研究一下相关的东西。